# Team Notebook

rizkwar

October 20, 2026

## Contents


1 01-PrecomputeEuler 2
2 02-KMP 2
3 03-Zfunction 2
4 04-AhoCorrasick 2
5 05-SuffixArray 3
6 06-String Hashing 4
7 07-HashingCustom 4
8 08-SCC 5
9 09-2SAT 5
10 10-LCA 6
11 11-HLD 6
12 12-CentroidDecomposition 7
13 13-EulerTourTree 7
14 14-CartesianTree 7
15 15-Persistent Segment Tree 7
16 16-LiChaoTree 8
17 17-PBDS 9
18 18-Mo'sAlgorithm 9
19 19-ParallelBinser 9
20 20-Maxflow 9
21 21-MCMF 10
22 22-EdmondBlossom 11
23 23-Hungarian 12
24 24-DPBrokenProfile 12
25 25-DPSOS 13
26 26-FFT 13
27 27-NTT 13
28 28-FWHT 14
29 29-prefactorial 14
30 30-Random 15
31 31-GEOM-PointVector 15
32 32-GEOM-LineSegment 15
33 33-GEOM-Circle 16
34 34-GEOM-Polygon 16
35 35-Treap 18

# 1 01-PrecomputeEuler

```
vector<int> phi(n + 2);
for(int i = 1; i <= n; i++) phi[i] = i;

for(int i = 2; i <= n; i++){
    if(phi[i] == i){
        for(int j = 1; j <= n; j += i){
            phi[j] -= phi[j] / i;
        }
    }
}
```

# 2 02-KMP

```
struct KMP {
    string p;
    vector<int> pi;
    KMP(const string& p) : p(p), pi(p.size()) {
        for (int i = 1, j = 0; i < (int)p.size(); i++) {
            while (j && p[i] != p[j])
                j = pi[j - 1];

            if (p[i] == p[j])
                j++;

            pi[i] = j;
        }
    }

    int count(const string& s) {
        int ans = 0;
        for (int i = 0, j = 0; i < (int)s.size(); i++) {
            while (j && s[i] != p[j])
                j = pi[j - 1];

            if (s[i] == p[j])
                j++;

            if (j == (int)p.size()) {
                ans++;
                j = pi[j - 1];
            }
        }
        return ans;
    }
};
```

# 3 03-Zfunction

```
vector<int> zFunction(const string& s) {
    int n = s.size();
    vector<int> z(n);

    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i <= r) {
            z[i] = min(r - i + 1, z[i - l]);
        }

        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }

        if (i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }

    return z;
}

vector<int> search(const string& text, const string& pattern
    ) {
    string s = pattern + '#' + text;
    vector<int> z = zFunction(s);
    vector<int> pos;

    int m = pattern.size();

    for (int i = m + 1; i < (int)s.size(); i++) {
        if (z[i] == m) {
            pos.push_back(i - m - 1);
        }
    }

    return pos;
}
```

# 4 04-AhoCorrasick

```
struct AhoCorasick {
    static constexpr int ALP = 52;

    vector<array<int, ALP>> nxt;
    vector<int> fail;
    vector<vector<int>> ids;
    vector<int> seen;
    vector<int> order;

    AhoCorasick(int maxNodes) {
        nxt.reserve(maxNodes);
        fail.reserve(maxNodes);
        ids.reserve(maxNodes);
        seen.reserve(maxNodes);

        addNode(); // root = 0
    }

    int addNode() {
        nxt.push_back({});
        nxt.back().fill(-1);
        fail.push_back(0);
        ids.push_back({});
        seen.push_back(0);
        return (int)nxt.size() - 1;
    }

    int getChar(char c) {
        return islower(c) ? c - 'a' : c - 'A' + 26;
    }

    void addWord(const string& s, int id) {
        int u = 0;

        for (char c : s) {
            int x = getChar(c);

            if (nxt[u][x] == -1)
                nxt[u][x] = addNode();

            u = nxt[u][x];
        }

        ids[u].push_back(id);
    }

    void build() {
        queue<int> q;

        for (int c = 0; c < ALP; c++) {
            if (nxt[0][c] != -1) {
                q.push(nxt[0][c]);
            } else {
                nxt[0][c] = 0;
```

```
            }
        }

        while (!q.empty()) {
            int u = q.front();
            q.pop();
            order.push_back(u);

            for (int c = 0; c < ALP; c++) {
                int v = nxt[u][c];

                if (v == -1) {
                    nxt[u][c] = nxt[fail[u]][c];
                } else {
                    fail[v] = nxt[fail[u]][c];
                    q.push(v);
                }
            }
        }
    }

    vector<int> solve(const string& s, int q) {
        int u = 0;

        for (char c : s) {
            u = nxt[u][getChar(c)];
            seen[u] = 1;
        }

        // Propagate occurrence information through failure
            links.
        for (int i = (int)order.size() - 1; i >= 0; i--) {
            int u = order[i];
            seen[fail[u]] |= seen[u];
        }

        vector<int> ans(q + 1);

        for (int u = 0; u < (int)nxt.size(); u++) {
            if (!seen[u]) continue;

            for (int id : ids[u])
                ans[id] = 1;
        }

        return ans;
    }
};

void solve() {
    string s;
    cin >> s;

    int q;
    cin >> q;

    vector<string> qry(q + 1);
    int totalLen = 0;

    for (int i = 1; i <= q; i++) {
        cin >> qry[i];
        totalLen += qry[i].size();
    }

    AhoCorasick aho(totalLen + 1);

    for (int i = 1; i <= q; i++)
        aho.addWord(qry[i], i);

    aho.build();

    auto ans = aho.solve(s, q);

    for (int i = 1; i <= q; i++)
        cout << (ans[i] ? "y\n" : "n\n");
}
```

# 5 05-SuffixArray

```
struct SuffixArray {
    string s;
    int n;
    vector<int> sa, c, lcp;

    SuffixArray(string s) {
        this->s = s + "$";
        n = s.size() + 1;
        buildSA();
    }

    void buildSA() {
        sa.resize(n);
        c.resize(n);

        vector<pair<char, int>> a(n);
        for (int i = 0; i < n; i++) {
            a[i] = {s[i], i};
        }

        sort(a.begin(), a.end());

        for (int i = 0; i < n; i++) {
            sa[i] = a[i].second;
        }

        c[sa[0]] = 0;
        for (int i = 1; i < n; i++) {
            c[sa[i]] = c[sa[i - 1]] + (a[i].first != a[i -
                1].first);
        }

        for (int k = 0; (1 << k) < n; k++) {
            vector<int> nsa(n), cnt(n), pos(n);

            for (int i = 0; i < n; i++) {
                nsa[i] = (sa[i] - (1 << k) + n) % n;
            }

            for (int x : nsa) {
                cnt[c[x]]++;
            }

            for (int i = 1; i < n; i++) {
                pos[i] = pos[i - 1] + cnt[i - 1];
            }

            for (int x : nsa) {
                nsa[pos[c[x]]++] = x;
            }

            sa = nsa;

            vector<int> nc(n);
            for (int i = 1; i < n; i++) {
                pair<int, int> x = {
                    c[sa[i]],
                    c[(sa[i] + (1 << k)) % n]
                };
                pair<int, int> y = {
                    c[sa[i - 1]],
                    c[(sa[i - 1] + (1 << k)) % n]
                };

                nc[sa[i]] = nc[sa[i - 1]] + (x != y);
            }

            c = nc;
        }
```

```
    }

    void buildLCP() {
        lcp.assign(n, 0);
        vector<int> rank(n);

        for (int i = 0; i < n; i++) {
            rank[sa[i]] = i;
        }

        for (int i = 0, k = 0; i < n - 1; i++) {
            int r = rank[i];

            if (r == 0) {
                continue;
            }

            int j = sa[r - 1];

            while (s[i + k] == s[j + k]) {
                k++;
            }

            lcp[r] = k;

            if (k) {
                k--;
            }
        }
    }

    bool contains(const string& t) {
        int l = 0, r = n - 1;

        while (l <= r) {
            int mid = (l + r) >> 1;
            int cmp = s.compare(sa[mid], t.size(), t);

            if (cmp == 0) {
                return true;
            }

            if (cmp < 0) {
                l = mid + 1;
            } else {
                r = mid - 1;
            }
        }

        return false;
    }
};
```

# 6 06-String Hashing

```
struct Hash {
    using ull = unsigned long long;

    const ull B1 = 911382323, B2 = 972663749;
    const ull M1 = 1000000007, M2 = 1000000009;

    string s;
    int n;
    vector<ull> h1, h2, p1, p2;

    Hash(string s) {
        this->s = s;
        n = s.size();

        h1.resize(n + 1);
        h2.resize(n + 1);
        p1.resize(n + 1);
        p2.resize(n + 1);

        p1[0] = p2[0] = 1;

        for (int i = 0; i < n; i++) {
            int x = s[i] - 'a' + 1;

            h1[i + 1] = (h1[i] * B1 + x) % M1;
            h2[i + 1] = (h2[i] * B2 + x) % M2;

            p1[i + 1] = p1[i] * B1 % M1;
            p2[i + 1] = p2[i] * B2 % M2;
        }
    }

    pair<ull, ull> get(int l, int r) {
        ull x1 = (h1[r + 1] - h1[l] * p1[r - l + 1] % M1 + M1
            ) % M1;
        ull x2 = (h2[r + 1] - h2[l] * p2[r - l + 1] % M2 + M2
            ) % M2;
        return {x1, x2};
    }

    bool equal(int l1, int r1, int l2, int r2) {
        return get(l1, r1) == get(l2, r2);
    }
};
```

# 7 07-HashingCustom

```
#include <bits/stdc++.h>
using namespace std;
typedef long double ld;
#define nel cout<<"\n"
#define f first
#define s second
#define int long long
mt19937_64 rd(chrono::steady_clock::now().time_since_epoch()
    .count());

// --- compact custom hash (splitmix64) ---
struct custom_hash {
    using ull = unsigned long long;
    static ull splitmix64(ull x) {
        x += 0x9e3779b97f4a7c15ULL;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
        return x ^ (x >> 31);
    }
    size_t operator()(unsigned long long x) const {
        static const ull FIXED_RANDOM = chrono::steady_clock
            ::now().time_since_epoch().count();
        return (size_t)splitmix64(x + FIXED_RANDOM);
    }
};

// --- usage ---
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    unordered_map<long long,int,custom_hash> mp;
    mp.reserve(1024);            // reserve before heavy
        inserts
    mp.max_load_factor(0.25f);   // reduce collisions /
        rehashing pressure

    mp[1] = 10;
    if (mp.find(1) != mp.end()) cout << mp[1] << '\n';
}
```

# 8 08-SCC

```
struct SCC {
    int n, cnt;
    vector<vector<int>> g, rg;
    vector<int> comp, order, deg;
    vector<bool> vis;

    SCC(int n) {
        this->n = n;
        cnt = 0;
        g.assign(n + 1, {});
        rg.assign(n + 1, {});
        comp.assign(n + 1, -1);
        vis.assign(n + 1, false);
    }

    void addEdge(int node, int next) {
        g[node].push_back(next);
        rg[next].push_back(node);
    }

    void dfs1(int node) {
        vis[node] = true;

        for (int next : g[node]) {
            if (!vis[next]) {
                dfs1(next);
            }
        }

        order.push_back(node);
    }

    void dfs2(int node, int c) {
        comp[node] = c;

        for (int next : rg[node]) {
            if (comp[next] == -1) {
                dfs2(next, c);
            }
        }
    }

    void run() {
        for (int node = 1; node <= n; node++) {
            if (!vis[node]) {
                dfs1(node);
            }
        }

        for (int i = n - 1; i >= 0; i--) {
            if (comp[order[i]] == -1) {
                dfs2(order[i], cnt);
                cnt++;
            }
        }
    }

    vector<vector<int>> buildDAG() {
        vector<vector<int>> dag(cnt);
        vector<set<int>> seen(cnt);
        deg.assign(cnt, 0);

        for (int node = 1; node <= n; node++) {
            for (int next : g[node]) {
                int a = comp[node];
                int b = comp[next];

                if (a != b && !seen[a].count(b)) {
                    seen[a].insert(b);
                    dag[a].push_back(b);
                    deg[b]++;
                }
            }
        }

        return dag;
    }
};
```

# 9 09-2SAT

```
struct TwoSat {
    int n;
    vector<vector<int>> g, rg;
    vector<int> comp, order;
    vector<bool> vis;

    TwoSat(int n) {
        this->n = n;
        g.assign(2 * n, {});
        rg.assign(2 * n, {});
        comp.assign(2 * n, -1);
        vis.assign(2 * n, false);
    }

    // i = true: 2*i, i = false: 2*i+1
    // negation: x ^ 1

    // clause (a OR b)
    void add(int a, int b) {
        g[a ^ 1].push_back(b);
        g[b ^ 1].push_back(a);
        rg[b].push_back(a ^ 1);
        rg[a].push_back(b ^ 1);
    }

    void dfs1(int node) {
        vis[node] = true;

        for (int next : g[node]) {
            if (!vis[next]) {
                dfs1(next);
            }
        }

        order.push_back(node);
    }

    void dfs2(int node, int c) {
        comp[node] = c;

        for (int next : rg[node]) {
            if (comp[next] == -1) {
                dfs2(next, c);
            }
        }
    }

    bool solve(vector<int>& ans) {
        for (int i = 0; i < 2 * n; i++) {
            if (!vis[i]) {
                dfs1(i);
            }
        }

        reverse(order.begin(), order.end());

        int c = 0;
        for (int node : order) {
            if (comp[node] == -1) {
                dfs2(node, c++);
            }
        }

        ans.resize(n);
```

```
        for (int i = 0; i < n; i++) {
            if (comp[2 * i] == comp[2 * i + 1]) {
                return false;
            }
            ans[i] = comp[2 * i] > comp[2 * i + 1];
        }

        return true;
    }
};
```

# 10 10-LCA

```
const int LOG = 20;

vector<vector<int>> adj;
int up[maxn][LOG + 1], dep[maxn];

void dfs(int node, int p) {
    up[node][0] = p;

    for (int j = 1; j <= LOG; j++) {
        up[node][j] = up[up[node][j - 1]][j - 1];
    }

    for (int next : adj[node]) {
        if (next != p) {
            dep[next] = dep[node] + 1;
            dfs(next, node);
        }
    }
}

int lca(int node, int next) {
    if (dep[node] < dep[next]) {
        swap(node, next);
    }

    int diff = dep[node] - dep[next];

    for (int j = 0; j <= LOG; j++) {
        if (diff & (1 << j)) {
            node = up[node][j];
        }
    }

    if (node == next) {
        return node;
    }

    for (int j = LOG; j >= 0; j--) {
        if (up[node][j] != up[next][j]) {
            node = up[node][j];
            next = up[next][j];
        }
    }

    return up[node][0];
}
```

# 11 11-HLD

```
struct HLD {
    int n, timer;
    vector<vector<int>> adj;
    vector<int> par, dep, sz, head, pos;

    HLD(int n = 0) {
        init(n);
    }

    void init(int n) {
        this->n = n;
        timer = 0;
        adj.assign(n + 1, {});
        par.resize(n + 1);
        dep.resize(n + 1);
        sz.resize(n + 1);
        head.resize(n + 1);
        pos.resize(n + 1);
    }

    void addEdge(int node, int next) {
        adj[node].push_back(next);
        adj[next].push_back(node);
    }

    void dfs(int node, int p) {
        par[node] = p;
        sz[node] = 1;

        for (int next : adj[node]) {
            if (next == p) continue;

            dep[next] = dep[node] + 1;
            dfs(next, node);
            sz[node] += sz[next];
        }
    }

    void decompose(int node) {
        pos[node] = ++timer;

        int heavy = -1;
        for (int next : adj[node]) {
            if (next == par[node]) continue;

            if (heavy == -1 || sz[next] > sz[heavy])
                heavy = next;
        }

        if (heavy == -1) return;

        head[heavy] = head[node];
        decompose(heavy);

        for (int next : adj[node]) {
            if (next == par[node] || next == heavy) continue;

            head[next] = next;
            decompose(next);
        }
    }

    void build(int root = 1) {
        dep[root] = 0;
        dfs(root, 0);

        timer = 0;
        head[root] = root;
        decompose(root);
    }

    int lca(int node, int next) {
        while (head[node] != head[next]) {
            if (dep[head[node]] > dep[head[next]])
                node = par[head[node]];
            else
                next = par[head[next]];
        }

        return dep[node] < dep[next] ? node : next;
    }
};
```

# 12 12-CentroidDecomposition

```
struct CentroidDecomposition {
  int n;
  vector<vector<int>> g;
  vector<int> sz;
  vector<bool> dead;

  CentroidDecomposition(int n) : n(n) {
    g.resize(n + 1);
    sz.resize(n + 1);
    dead.assign(n + 1, false);
  }

  void add_edge(int node, int next) {
    g[node].push_back(next);
    g[next].push_back(node);
  }

  int get_size(int node, int p) {
    sz[node] = 1;

    for (int next : g[node]) {
      if (next == p || dead[next]) continue;
      sz[node] += get_size(next, node);
    }

    return sz[node];
  }

  int get_centroid(int node, int p, int tot) {
    for (int next : g[node]) {
      if (next == p || dead[next]) continue;

      if (sz[next] > tot / 2)
        return get_centroid(next, node, tot);
    }

    return node;
  }

  int get_centroid(int start) {
    int tot = get_size(start, -1);
    return get_centroid(start, -1, tot);
  }

  void collect(int node, int p, int dist, vector<int>& nodes
      ) {
    nodes.push_back(dist);

    for (int next : g[node]) {
      if (next == p || dead[next]) continue;
      collect(next, node, dist + 1, nodes);
    }
  }

  void decompose(int node) {
    int c = get_centroid(node);

    dead[c] = true;

    // PROCESS CENTROID c HERE

    for (int next : g[c]) {
      if (dead[next]) continue;
      decompose(next);
    }
  }
};
```

# 13 13-EulerTourTree

```
vector<ll>adj[maxn],flat(2*maxn);
ll st[maxn], en[maxn];

ll t;
void dfs(ll node, ll par) {
    flat[++t] = node;
    st[node] = t;
    for (auto next : adj[node]) if (next != par) {
        dfs(next,node);
    }
    flat[++t] = node;
    en[node] = t;
}
```

# 14 14-CartesianTree

```
struct CartesianTree {
    int root;
    vector<int> l, r, par;

    CartesianTree(const vector<int>& a) {
        int n = a.size();
        l.assign(n, -1);
        r.assign(n, -1);
        par.assign(n, -1);

        vector<int> st;

        for (int i = 0; i < n; i++) {
            int last = -1;

            while (!st.empty() && a[st.back()] > a[i]) {
                last = st.back();
                st.pop_back();
            }

            if (last != -1) {
                l[i] = last;
                par[last] = i;
            }

            if (!st.empty()) {
                r[st.back()] = i;
                par[i] = st.back();
            }

            st.push_back(i);
        }

        root = st.front();
    }
};
```

# 15 15-Persistent Segment Tree

```
int st[40 * maxn], L[40 * maxn], R[40 * maxn];
int a[maxn], root[maxn];
int ptr = 0;

int build(int l, int r) {
    int node = ++ptr;

    if (l == r) {
        st[node] = a[l];
        return node;
    }

    int mid = (l + r) >> 1;
    L[node] = build(l, mid);
    R[node] = build(mid + 1, r);
    st[node] = st[L[node]] + st[R[node]];
```

```
    return node;
}

int qry(int node, int l, int r, int x, int y) {
    if (l > y || r < x) {
        return 0;
    }

    if (l >= x && r <= y) {
        return st[node];
    }

    int mid = (l + r) >> 1;
    return qry(L[node], l, mid, x, y)
         + qry(R[node], mid + 1, r, x, y);
}

int upd(int prev, int l, int r, int pos, int val) {
    int node = ++ptr;

    st[node] = st[prev];
    L[node] = L[prev];
    R[node] = R[prev];

    if (l == r) {
        st[node] = val;
        return node;
    }

    int mid = (l + r) >> 1;

    if (pos <= mid) {
        L[node] = upd(L[prev], l, mid, pos, val);
    } else {
        R[node] = upd(R[prev], mid + 1, r, pos, val);
    }

    st[node] = st[L[node]] + st[R[node]];
    return node;
}

void solve() {
    cin >> n >> q;

    for (int i = 1; i <= n; i++) {
        cin >> a[i];
    }

    root[1] = build(1, n);

    vector<int> v(q + 3);
    v[1] = 1;

    int idx = 1, cur = 1;

    for (int i = 1; i <= q; i++) {
        int mode, k;
        cin >> mode >> k;

        if (mode == 1) {
            int pos, val;
            cin >> pos >> val;

            root[++idx] = upd(root[v[k]], 1, n, pos, val);
            v[k] = idx;
        } else if (mode == 2) {
            int l, r;
            cin >> l >> r;

            cout << qry(root[v[k]], 1, n, l, r) << '\n';
        } else {
            root[++idx] = root[v[k]];
            cur++;
            v[cur] = idx;
        }
    }
}
```

# 16 16-LiChaoTree

```
struct LiChao {
  struct line {
    int m, b;
    int eval(int x) const { return m * x + b; }
  };
  int L, R;
  const int NEG_INF = -4e18;
  vector<line> seg;
  vector<int> lc;
  vector<int> rc;
  int nodes;
  void init(int _L, int _R){
    L = _L; R = _R;
    nodes = 1;
    seg.assign(1, {0, NEG_INF});
    lc.assign(1, -1);
    rc.assign(1, -1);
  }
  int new_node(){
    seg.push_back({0, NEG_INF});
    lc.push_back(-1);
    rc.push_back(-1);
    return nodes++;
  }
  void add_line(int nm, int nb){ add_line_rec(0, L, R, {nm,
       nb}); }
  void add_line_rec(int idx, int l, int r, line nw){
    if(seg[idx].b == NEG_INF){
      seg[idx] = nw;
      return;
    }
    int mid = (l + r) >> 1;
    bool left = nw.eval(l) >= seg[idx].eval(l);
    bool mpos = nw.eval(mid) >= seg[idx].eval(mid);
    if(mpos){
      swap(seg[idx], nw);
    }
    if(l == r) return;
    if(left != mpos){
      if(lc[idx] == -1){
        int t = new_node();
        lc[idx] = t;
      }
      add_line_rec(lc[idx], l, mid, nw);
    } else {
      if(rc[idx] == -1){
        int t = new_node();
        rc[idx] = t;
      }
      add_line_rec(rc[idx], mid+1, r, nw);
    }
  }
  int query(int x){ return query_rec(0, L, R, x); }
  int query_rec(int idx, int l, int r, int x){
    if(idx == -1) return NEG_INF;
    int cur = seg[idx].eval(x);
    if(l == r) return cur;
    int mid = (l + r) >> 1;
    int oth = NEG_INF;
    if(x <= mid){
      if(lc[idx] != -1) oth = query_rec(lc[idx], l, mid, x);
    } else {
      if(rc[idx] != -1) oth = query_rec(rc[idx], mid+1, r, x)
           ;
    }
    return max(cur, oth);
  }
```

```
};
```

# 17 17-PBDS

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template <typename T>
using ordered_set = tree<T,null_type,less<T>,rb_tree_tag,
    tree_order_statistics_node_update>;

os.find_by_order(x); // mengambil index ke x
os.order_of_key(x); // berapa banyak elemen yang < x
```

# 18 18-Mo'sAlgorithm

```
struct Mo {
    int n, q, B;
    vector<int> a, freq;
    long long cur;

    Mo(vector<int> a) : a(a) {
        n = a.size() - 1;
        B = sqrt(n);
        freq.assign(1000005, 0);
        cur = 0;
    }

    void add(int x) {
        cur -= 1LL * x * freq[x] * freq[x];
        freq[x]++;
        cur += 1LL * x * freq[x] * freq[x];
    }

    void remove(int x) {
        cur -= 1LL * x * freq[x] * freq[x];
        freq[x]--;
        cur += 1LL * x * freq[x] * freq[x];
    }

    vector<long long> solve(vector<pair<int,int>> qry) {
        q = qry.size();

        vector<tuple<int,int,int>> v(q);
        for (int i = 0; i < q; i++) {
            v[i] = {qry[i].first, qry[i].second, i};
        }

        sort(v.begin(), v.end(), [&](auto &x, auto &y) {
            auto [l1, r1, id1] = x;
            auto [l2, r2, id2] = y;

            int b1 = l1 / B, b2 = l2 / B;
            if (b1 != b2) {
                return b1 < b2;
            }
            return (b1 & 1) ? r1 > r2 : r1 < r2;
        });

        vector<long long> ans(q);
        int l = 1, r = 0;

        for (auto [ql, qr, id] : v) {
            while (r < qr) {
                add(a[++r]);
            }
            while (l > ql) {
                add(a[--l]);
            }
            while (r > qr) {
                remove(a[r--]);
            }
            while (l < ql) {
                remove(a[l++]);
            }

            ans[id] = cur;
        }

        return ans;
    }
};
```

# 19 19-ParallelBinser

```
for (int i = 1; i <= q; i++) {
    cin >> b[i].first >> b[i].second;
    lo[i] = 1;
    hi[i] = m;
    ans[i] = -1;
}
while (true) {
    bool done = true;
    vector<vector<int>> at(m + 1);

    for (int i = 1; i <= q; i++) {
        if (lo[i] <= hi[i]) {
            done = false;
            int mid = (lo[i] + hi[i]) >> 1;
            at[mid].push_back(i);
        }
    }

    if (done) {
        break;
    }

    iota(par + 1, par + n + 1, 1);
    for (int i = 1; i <= m; i++) {
        merge(a[i].first, a[i].second);
        for (int id : at[i]) {
            if (same(b[id].first, b[id].second)) {
                ans[id] = i;
                hi[id] = i - 1;
            } else {
                lo[id] = i + 1;
            }
        }
    }
}

for (int i = 1; i <= q; i++) {
    if (b[i].first == b[i].second) {
        ans[i] = 0;
    }
    cout << ans[i] << '\n';
}
```

# 20 20-Maxflow

```
struct Edge {
    int to, rev, cap;
    bool orig;
};

struct Dinic {
    int n;
    vector<vector<Edge>> g;
    vector<int> lv, ptr;
```

```
    Dinic(int n = 0) { init(n); }

    void init(int n) {
        this->n = n;
        g.assign(n, {});
        lv.resize(n);
        ptr.resize(n);
    }

    void addEdge(int node, int next, int cap) {
        g[node].push_back({next, (int)g[next].size(), cap,
             1});
        g[next].push_back({node, (int)g[node].size() - 1, 0,
             0});
    }

    bool bfs(int s, int t) {
        fill(lv.begin(), lv.end(), -1);
        queue<int> q;
        q.push(s);
        lv[s] = 0;

        while (!q.empty()) {
            int node = q.front();
            q.pop();

            for (auto &e : g[node]) {
                if (e.cap > 0 && lv[e.to] == -1) {
                    lv[e.to] = lv[node] + 1;
                    q.push(e.to);
                }
            }
        }
        return lv[t] != -1;
    }

    int dfs(int node, int t, int f) {
        if (node == t) { return f; }

        for (int &i = ptr[node]; i < (int)g[node].size(); i
             ++) {
            auto &e = g[node][i];

            if (e.cap == 0 || lv[e.to] != lv[node] + 1) {
                continue;
            }

            int x = dfs(e.to, t, min(f, e.cap));
            if (x > 0) {
                e.cap -= x;
                g[e.to][e.rev].cap += x;
                return x;
            }
        }
        return 0;
    }

    int flow(int s, int t) {
        int ans = 0;

        while (bfs(s, t)) {
            fill(ptr.begin(), ptr.end(), 0);

            while (true) {
                int f = dfs(s, t, 1e9);
                if (f == 0) { break; }
                ans += f;
            }
        }
        return ans;
    }

    bool recover(int node, int t, vector<int>& path, vector<
         bool>& vis) {
        if (node == t) { return true; }
        vis[node] = true;

        for (auto &e : g[node]) {
            if (!e.orig || vis[e.to] || g[e.to][e.rev].cap ==
                 0) {
                continue;
            }

            g[e.to][e.rev].cap--;
            path.push_back(e.to);

            if (recover(e.to, t, path, vis)) {
                return true;
            }

            path.pop_back();
            g[e.to][e.rev].cap++;
        }

        return false;
    }
};
```

# 21 21-MCMF

```
struct Edge {
    int to, rev, cap, cost, orig;
};

struct MinCostFlow {
    int n;
    vector<vector<Edge>> g;
    vector<int> pi, dist, pv_node, pv_edge;

    MinCostFlow(int n = 0) {
        init(n);
    }

    void init(int n) {
        this->n = n;
        g.assign(n, {});
        pi.assign(n, 0);
        dist.resize(n);
        pv_node.resize(n);
        pv_edge.resize(n);
    }

    void addEdge(int node, int next, int cap, int cost) {
        g[node].push_back({next, (int)g[next].size(), cap,
             cost, cap});
        g[next].push_back({node, (int)g[node].size() - 1, 0,
             -cost, 0});
    }

    bool sp(int s, int t) {
        const int INF = 1e9;
        priority_queue<pair<int,int>, vector<pair<int,int>>,
                       greater<pair<int,int>>> pq;

        fill(dist.begin(), dist.end(), INF);
        fill(pv_node.begin(), pv_node.end(), -1);
        fill(pv_edge.begin(), pv_edge.end(), -1);

        dist[s] = 0;
        pq.push({0, s});

        while (!pq.empty()) {
            auto [d, node] = pq.top();
            pq.pop();

            if (d != dist[node]) {
                continue;
            }
```

```
        for (int i = 0; i < (int)g[node].size(); i++) {
            auto &e = g[node][i];
            if (e.cap == 0) {
                continue;
            }

            int next = e.to;
            int nd = d + e.cost + pi[node] - pi[next];

            if (nd < dist[next]) {
                dist[next] = nd;
                pv_node[next] = node;
                pv_edge[next] = i;
                pq.push({nd, next});
            }
        }
    }

    return dist[t] != INF;
}

pair<int,int> flow(int s, int t, int need) {
    int f = 0, cost = 0;
    const int INF = 1e9;

    while (f < need && sp(s, t)) {
        for (int i = 0; i < n; i++) {
            if (dist[i] < INF) {
                pi[i] += dist[i];
            }
        }

        int add = need - f;

        for (int next = t; next != s; next = pv_node[next
            ]) {
            add = min(add, g[pv_node[next]][pv_edge[next
                ]].cap);
        }

        int pathCost = pi[t] - pi[s];

        for (int next = t; next != s; next = pv_node[next
            ]) {
            int node = pv_node[next], i = pv_edge[next];
            auto &e = g[node][i];

            e.cap -= add;
            g[next][e.rev].cap += add;

            }

            f += add;
            cost += add * pathCost;
        }

        return {f, cost};
    }
};

// kalo mau diubah jadi maximual yang diubah cost nya jadi -
    cost
// -> return {f, -cost}
// trus pas add edge weight nya juga dibikin -weight
```

# 22 22-EdmondBlossom

```
struct Blossom {
    int n;
    vector<vector<int>> g;
    vector<int> match, p, base;
    vector<bool> used, blossom;
    queue<int> q;

    Blossom(int n = 0) {
        init(n);
    }

    void init(int n) {
        this->n = n;
        g.assign(n, {});
        match.assign(n, -1);
        p.resize(n);
        base.resize(n);
        used.resize(n);
        blossom.resize(n);
    }

    void addEdge(int node, int next) {
        g[node].push_back(next);
        g[next].push_back(node);
    }

    int lca(int a, int b) {
        vector<bool> seen(n);

        while (true) {
            a = base[a];
            seen[a] = true;

            if (match[a] == -1) {
                break;
            }

            a = p[match[a]];
        }

        while (true) {
            b = base[b];

            if (seen[b]) {
                return b;
            }

            b = p[match[b]];
        }
    }

    void markPath(int node, int b, int child) {
        while (base[node] != b) {
            blossom[base[node]] = true;
            blossom[base[match[node]]] = true;
            p[node] = child;
            child = match[node];
            node = p[match[node]];
        }
    }

    bool findPath(int root) {
        fill(used.begin(), used.end(), false);
        fill(p.begin(), p.end(), -1);
        iota(base.begin(), base.end(), 0);

        while (!q.empty()) {
            q.pop();
        }

        q.push(root);
        used[root] = true;

        while (!q.empty()) {
            int node = q.front();
            q.pop();

            for (int next : g[node]) {
                if (base[node] == base[next] || match[node] ==
                    next) {
                    continue;
```

```
        }

        if (next == root || (match[next] != -1 && p[
             match[next]] != -1)) {
            int b = lca(node, next);
            fill(blossom.begin(), blossom.end(), false
                 );

            markPath(node, b, next);
            markPath(next, b, node);

            for (int i = 0; i < n; i++) {
                if (blossom[base[i]]) {
                    base[i] = b;

                    if (!used[i]) {
                        used[i] = true;
                        q.push(i);
                    }
                }
            }
        } else if (p[next] == -1) {
            p[next] = node;

            if (match[next] == -1) {
                int cur = next;

                while (cur != -1) {
                    int prev = p[cur];
                    int nextMatch = (prev == -1 ? -1 :
                         match[prev]);

                    match[cur] = prev;

                    if (prev != -1) {
                        match[prev] = cur;
                    }

                    cur = nextMatch;
                }

                return true;
            }

            used[match[next]] = true;
            q.push(match[next]);
        }
    }
}
```

```
        return false;
    }

    int solve() {
        int ans = 0;

        for (int i = 0; i < n; i++) {
            if (match[i] == -1 && findPath(i)) {
                ans++;
            }
        }

        return ans;
    }
};
```

# 23 23-Hungarian

```
struct Hungarian {
    int n;
    vector<vector<int>> a;
    vector<int> node, next, p, way;

    Hungarian(const vector<vector<int>>& a) : a(a) {
        n = (int)a.size() - 1;
        node.assign(n + 1, 0);
        next.assign(n + 1, 0);
        p.assign(n + 1, 0);
        way.assign(n + 1, 0);
    }

    pair<int, vector<int>> solve() {
        const int INF = 1e9;

        for (int i = 1; i <= n; i++) {
            p[0] = i;

            int j0 = 0;
            vector<int> minv(n + 1, INF);
            vector<bool> used(n + 1, false);

            do {
                used[j0] = true;
                int i0 = p[j0];
                int delta = INF, j1 = 0;

                for (int j = 1; j <= n; j++) {
                    if (used[j]) continue;

                    int cur = a[i0][j] - node[i0] - next[j];

                    if (cur < minv[j]) {
                        minv[j] = cur;
                        way[j] = j0;
                    }

                    if (minv[j] < delta) {
                        delta = minv[j];
                        j1 = j;
                    }
                }

                for (int j = 0; j <= n; j++) {
                    if (used[j]) {
                        node[p[j]] += delta;
                        next[j] -= delta;
                    } else {
                        minv[j] -= delta;
                    }
                }

                j0 = j1;
            } while (p[j0] != 0);

            do {
                int j1 = way[j0];
                p[j0] = p[j1];
                j0 = j1;
            } while (j0 != 0);
        }

        vector<int> match(n + 1);
        for (int j = 1; j <= n; j++)
            match[p[j]] = j;

        return {-next[0], match};
    }
};
```

# 24 24-DPBrokenProfile

```
int r, c, m, b;
cin >> r >> c >> m >> b;

if (c > 6) swap(r, c);
```

```
int n = 2 * c;
int mask = (1 << n) - 1;

vector<vector<int>> dp(1 << n, vector<int>(m + 1));
dp[0][0] = 1;

for (int i = 0; i < r; i++) {
    for (int j = 0; j < c; j++) {
        vector<vector<int>> ndp(1 << n, vector<int>(m + 1));

        for (int used = 0; used < (1 << n); used++) {
            for (int cnt = 0; cnt <= m; cnt++) {
                if (!dp[used][cnt]) continue;

                int nxt = (used << 1) & mask;

                ndp[nxt][cnt] = (ndp[nxt][cnt] + dp[used][cnt
                    ]) % modn;

                if (cnt == m) continue;

                bool ok = true;

                if (j >= 1 && (used & 1)) ok = false;
                if (j >= 2 && (used & 2)) ok = false;
                if (i >= 1 && (used & (1 << (c - 1)))) ok =
                     false;
                if (i >= 2 && (used & (1 << (n - 1)))) ok =
                     false;

                if (ok) {
                    ndp[nxt | 1][cnt + 1] =
                        (ndp[nxt | 1][cnt + 1] + dp[used][cnt])
                            % modn;
                }
            }
        }

        dp.swap(ndp);
    }
}
```

# 25 25-DPSOS

```
// Sum over submasks: dp[S] =  f[T], T    S
for (int bit = 0; bit < K; bit++)
    for (int mask = 0; mask < (1 << K); mask++)
        if (mask & (1 << bit))
            dp[mask] += dp[mask ^ (1 << bit)];

// Sum over supermasks: dp[S] =  f[T], T   S
for (int bit = 0; bit < K; bit++)
    for (int mask = 0; mask < (1 << K); mask++)
        if (mask & (1 << bit))
            dp[mask ^ (1 << bit)] += dp[mask];

// Recursive: enumerate all submasks
void add(int mask, int bit) {
    if (dp[mask] >= 2) return;
    if (bit < 0) return void(dp[mask]++);
    add(mask, bit - 1);
    if (mask & (1 << bit))
        add(mask ^ (1 << bit), bit - 1);
}
```

# 26 26-FFT

```
struct FFT {
  using cd = complex<double>;
  ld PI=acos(-1);
  void fft(vector<cd>& a, bool inv){
    int n = a.size();
    for(int i=1,j=0;i<n;i++){
      int bit = n>>1;
      for(; j&bit; bit>>=1) j ^= bit;
      j ^= bit;
      if(i<j) swap(a[i], a[j]);
    }
    for(int len=2; len<=n; len<<=1){
      double ang = 2*PI/len * (inv ? -1 : 1);
      cd wlen(cos(ang), sin(ang));
      for(int i=0;i<n;i+=len){
        cd w(1);
        for(int j=0;j<len/2;j++){
          cd u = a[i+j];
          cd v = a[i+j+len/2] * w;
          a[i+j] = u+v;a
          a[i+j+len/2] = u-v;
          w *= wlen;
        }
      }
    }
    if(inv) for(auto &x: a) x /= n;
  }
  vector<long long> multiply(const vector<long long>& A,
      const vector<long long>& B){
    int n = 1;
    while(n < (int)A.size() + (int)B.size()) n <<= 1;
    vector<cd> fa(n);
    for(size_t i=0;i<A.size();++i) fa[i] = cd((double)A[i],
        0);
    for(size_t i=0;i<B.size();++i) fa[i] += cd(0, (double)B[i
        ]);
    fft(fa, false);
    vector<cd> fa2(n);
    for(int i=0;i<n;i++){
      cd j = fa[(n-i)%n];
      cd a = (fa[i] + conj(j)) * cd(0.5, 0);
      cd b = (fa[i] - conj(j)) * cd(0, -0.5);
      fa2[i] = a * b;
    }
    fft(fa2, true);
    vector<long long> res(n);
    for(int i=0;i<n;i++) res[i] = (long long) llround(fa2[i].
        real());
    return res;
  }
} fft;
```

# 27 27-NTT

```
struct NTT {
    int mod = 998244353;
    int root = 3;

    int power(int a, int e) {
        int r = 1;

        while (e) {
            if (e & 1) {
                r = 1LL * r * a % mod;
            }
            a = 1LL * a * a % mod;
            e >>= 1;
        }

        return r;
    }

    void ntt(vector<int>& a, bool inv) {
        int n = a.size();

        for (int i = 1, j = 0; i < n; i++) {
            int bit = n >> 1;
```

```
    while (j & bit) {
        j ^= bit;
        bit >>= 1;
    }

    j ^= bit;

    if (i < j) {
        swap(a[i], a[j]);
    }
}

for (int len = 2; len <= n; len <<= 1) {
    int wlen = power(root, (mod - 1) / len);

    if (inv) {
        wlen = power(wlen, mod - 2);
    }

    for (int i = 0; i < n; i += len) {
        int w = 1;

        for (int j = 0; j < len / 2; j++) {
            int u = a[i + j];
            int v = 1LL * a[i + j + len / 2] * w % mod
                ;

            a[i + j] = u + v;
            if (a[i + j] >= mod) {
                a[i + j] -= mod;
            }

            a[i + j + len / 2] = u - v;
            if (a[i + j + len / 2] < 0) {
                a[i + j + len / 2] += mod;
            }

            w = 1LL * w * wlen % mod;
        }
    }
}

if (inv) {
    int inv_n = power(n, mod - 2);

    for (int &x : a) {
        x = 1LL * x * inv_n % mod;
    }
}
```

```
    }

    vector<int> multiply(vector<int> A, vector<int> B) {
        if (A.empty() || B.empty()) {
            return {};
        }

        int sz = A.size() + B.size() - 1;
        int n = 1;

        while (n < sz) {
            n <<= 1;
        }

        A.resize(n);
        B.resize(n);

        ntt(A, false);
        ntt(B, false);

        for (int i = 0; i < n; i++) {
            A[i] = 1LL * A[i] * B[i] % mod;
        }

        ntt(A, true);
        A.resize(sz);

        return A;
    }
} ntt;
```

# 28 28-FWHT

```
struct FWHT {
    int mod;

    FWHT(int mod) : mod(mod) {}

    void transform(vector<int>& a) {
        int n = a.size();

        for (int len = 1; len < n; len <<= 1) {
            for (int i = 0; i < n; i += len << 1) {
                for (int j = 0; j < len; j++) {
                    int u = a[i + j];
                    int v = a[i + j + len];

                    a[i + j] = (u + v) % mod;
                    a[i + j + len] = (u - v + mod) % mod;
                }
            }
        }
    }

    void inverse(vector<int>& a) {
        transform(a);

        int inv_n = modpow(a.size(), mod - 2);
        for (int& x : a)
            x = 1LL * x * inv_n % mod;
    }

    int modpow(int a, int e) {
        int res = 1;

        while (e) {
            if (e & 1)
                res = 1LL * res * a % mod;
            a = 1LL * a * a % mod;
            e >>= 1;
        }

        return res;
    }
};
```

# 29 29-prefactorial

```
int fac[maxn + 5], ifac[maxn + 5];

int C(int n, int k) {
    if (k < 0 || k > n) {
        return 0;
    }
    return 1LL * fac[n] * ifac[k] % modn * ifac[n - k] % modn
        ;
}

void pre() {
    fac[0] = 1;

    for (int i = 1; i <= maxn; i++) {
        fac[i] = 1LL * fac[i - 1] * i % modn;
    }

    ifac[maxn] = binpow(fac[maxn], modn - 2);
```

```
    for (int i = maxn; i >= 1; i--) {
        ifac[i - 1] = 1LL * ifac[i] * i % modn;
    }
}
```

# 30 30-Random

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    count());
uniform_int_distribution<ll>dist(l,r);
ll ran = dist(rng)
```

# 31 31-GEOM-PointVector

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using ld = long double;
using pll = pair<ll,ll>;

const ld EPS = 1e-9L;
const ld PI = acosl(-1.0L);
const ld INF = 1e18L;

inline int sign(ld x) { return (x > EPS) - (x < -EPS); }
inline bool eq(ld a, ld b) { return sign(a - b) == 0; }
inline ld sq(ld x) { return x * x; }

// 2D POINT / VECTOR

struct P {
    ld x, y;
    P(ld x=0, ld y=0): x(x), y(y) {}

    P operator+(const P& o) const { return {x+o.x,y+o.y}; }
    P operator-(const P& o) const { return {x-o.x,y-o.y}; }
    P operator*(ld t) const { return {x*t,y*t}; }
    P operator/(ld t) const { return {x/t,y/t}; }
    bool operator==(const P& o) const { return eq(x,o.x)&&eq(
        y,o.y); }

    bool operator<(const P& o) const {
        if (!eq(x,o.x)) return x<o.x;
        return y<o.y;
    }

    ld dot(const P& o) const { return x*o.x+y*o.y; }
    ld cross(const P& o) const { return x*o.y-y*o.x; }
    ld norm() const { return dot(*this); }
    ld abs() const { return sqrtl(norm()); }
    ld arg() const { return atan2l(y,x); }

    P unit() const { return *this/abs(); }
    P perp() const { return {-y,x}; }
    P rot(ld a) const { return {x*cosl(a)-y*sinl(a),x*sinl(a)
        +y*cosl(a)}; }
    P conj() const { return {x,-y}; }

    friend ostream& operator<<(ostream& os,const P& p) {
        return os<<"("<<p.x<<", "<<p.y<<")";
    }
};

ld dist(P a,P b) { return (a-b).abs(); }
ld dist2(P a,P b) { return (a-b).norm(); }

// polar-angle sort around origin (uses atan2, O(n log n))
bool polarCmp(const P& a,const P& b) {
    int ha=(a.y<-EPS||(sign(a.y)==0&&a.x<-EPS));
    int hb=(b.y<-EPS||(sign(b.y)==0&&b.x<-EPS));
    if (ha!=hb) return ha<hb;
    ld cr=a.cross(b);
    if (sign(cr)!=0) return cr>0;
    return a.norm()<b.norm();
}
```

# 32 32-GEOM-LineSegment

```
// LINES & SEGMENTS
struct Line {
    P p,d;

    // construct from two points
    Line(P a,P b): p(a),d(b-a) {}

    // construct from ax + by = c
    Line(ld a,ld b,ld c): p(b?P(0,c/b):P(c/a,0)),d(b?P(b,-a):
        P(0,1)) {}

    // signed distance (positive = left of directed line)
    ld sdist(P q) const { return d.cross(q-p); }

    // side: +1 left, -1 right, 0 on line
    int side(P q) const { return sign(sdist(q)); }

    // foot of perpendicular from q
    P foot(P q) const { return p+d*(d.dot(q-p)/d.norm()); }

    // reflect q over this line
    P reflect(P q) const { return q+(foot(q)-q)*2; }

    // point at parameter t
    P at(ld t) const { return p+d*t; }
};

// Distance: point to LINE
ld distPL(P q,Line l) { return fabsl(l.sdist(q))/l.d.abs();
    }

// Distance: point to SEGMENT [a,b]
ld distPS(P q,P a,P b) {
    if ((b-a).dot(q-a)<=EPS) return dist(q,a);
    if ((a-b).dot(q-b)<=EPS) return dist(q,b);
    return distPL(q,Line(a,b));
}

//  LINELINE  intersection (returns false if parallel)
bool lineIntersect(Line l1,Line l2,P& out) {
    ld cr=l1.d.cross(l2.d);
    if (sign(cr)==0) return false;
    out=l1.p+l1.d*(l2.d.cross(l1.p-l2.p)/cr);
    return true;
}

//  SEGMENTSEGMENT  intersection (proper = no endpoints)
bool segIntersectProper(P a,P b,P c,P d) {
    Line l1(a,b),l2(c,d);
    return l1.side(c)*l1.side(d)<0&&l2.side(a)*l2.side(b)<0;
}

//  SEGMENTSEGMENT  intersection (including endpoints /
    collinear)
bool onSeg(P p,P a,P b) {
    return sign((p-a).cross(b-a))==0&&sign((p-a).dot(b-a))
        >=0&&sign((p-b).dot(a-b))>=0;
}

bool segIntersect(P a,P b,P c,P d) {
    if (segIntersectProper(a,b,c,d)) return true;
    return onSeg(c,a,b)||onSeg(d,a,b)||onSeg(a,c,d)||onSeg(b,
        c,d);
```

```
}
```

# 33 33-GEOM-Circle

```
// CIRCLE
struct Circle {
    P c; ld r;
    Circle(P c,ld r): c(c),r(r) {}
};

// pointcircle relation: -1 inside, 0 on, +1 outside
int ptCircle(P p,Circle C) { return sign(dist(p,C.c)-C.r); }

// tangent lines from external point p to circle C
// returns 0 (inside), 1 (tangent), or 2 points of tangency
int tangentPoints(P p,Circle C,P& t1,P& t2) {
    ld d2=dist2(p,C.c),r2=sq(C.r);
    if (sign(d2-r2)<0) return 0;
    if (sign(d2-r2)==0) { t1=t2=Line(p,C.c).foot(p); return
        1; }
    ld a=acosl(C.r/sqrtl(d2));
    P u=(C.c-p).unit();
    t1=p+u.rot(a)*sqrtl(d2-r2);
    t2=p+u.rot(-a)*sqrtl(d2-r2);
    return 2;
}

// LINECIRCLE intersections (0, 1, or 2 points)
int lineCircle(Line l,Circle C,P& p1,P& p2) {
    P f=l.foot(C.c);
    ld d2=dist2(f,C.c),r2=sq(C.r);
    int s=sign(d2-r2);
    if (s>0) return 0;
    if (s==0) { p1=p2=f; return 1; }
    ld dt=sqrtl(r2-d2)/l.d.abs();
    p1=f+l.d*dt;
    p2=f-l.d*dt;
    return 2;
}

// CIRCLECIRCLE intersections (0,1,2 points; also handles
    containment)
int circleCircle(Circle A,Circle B,P& p1,P& p2) {
    ld d=dist(A.c,B.c);
    if (sign(d-(A.r+B.r))>0) return 0;
    if (sign(d-fabsl(A.r-B.r))<0) return 0;
    if (sign(d)==0) return 0;
    ld cos_a=(sq(d)+sq(A.r)-sq(B.r))/(2*d*A.r);
    cos_a=max((ld)-1,min((ld)1,cos_a));
    ld a=acosl(cos_a);
    P u=(B.c-A.c).unit();

    p1=A.c+u.rot(a)*A.r;
    p2=A.c+u.rot(-a)*A.r;
    return sign(a)==0?1:2;
}

// CIRCLECIRCLE common tangent lines (0..4)
// external: 2 lines, internal: 2 lines (cross), total
    depends on config
vector<Line> commonTangents(Circle A,Circle B) {
    vector<Line> res;
    ld d=dist(A.c,B.c);

    for (int s:{-1,1}) {
        ld rr=A.r-s*B.r;
        if (sq(rr)>sq(d)+EPS) continue;

        ld a=(sq(d)>sq(rr))?acosl(-rr/d):0;
        P u=(B.c-A.c).unit();

        for (int t:{-1,1}) {
            P n=u.rot(t*a);
            P p=A.c+n*A.r;
            P q=B.c+n*(s*B.r);
            res.emplace_back(p,q);
        }
    }
    return res;
}

// Arc length, sector area, segment area
ld arcLen(Circle C,ld angle) { return C.r*fabsl(angle); }
ld sectorArea(Circle C,ld angle) { return .5L*sq(C.r)*fabsl(
    angle); }

ld segArea(Circle C,ld angle) {
    angle=fabsl(angle);
    return .5L*sq(C.r)*(angle-sinl(angle));
}

// Area of intersection of two circles
ld circleIntersectArea(Circle A,Circle B) {
    ld d=dist(A.c,B.c);
    if (sign(d-(A.r+B.r))>=0) return 0;
    if (sign(d-fabsl(A.r-B.r))<=0) {
        ld r=min(A.r,B.r);
        return PI*sq(r);
    }

    auto ang=[&](ld r,ld other,ld dd) {
        return 2*acosl((sq(dd)+sq(r)-sq(other))/(2*r*dd));
    };

    ld a1=ang(A.r,B.r,d),a2=ang(B.r,A.r,d);
    return segArea(A,a1)+segArea(B,a2);
}

vector<P> lineCircleIntersect(Circle c,Line l) {
    P v=c.c-l.p;
    ld t=v.dot(l.d)/l.d.norm();
    P foot=l.p+l.d*t;
    ld h2=dist2(foot,c.c),r2=sq(c.r);

    if (sign(h2-r2)>0) return {};
    if (sign(h2-r2)==0) return {foot};

    ld h_dist=sqrtl(r2-h2);
    P offset=l.d.unit()*h_dist;
    return {foot-offset,foot+offset};
}

ld PtoAngle(Circle c,P p) {
    ld angle=atan2l(p.y-c.c.y,p.x-c.c.x);
    if (angle<0) angle+=2*PI;
    return angle;
}

ld segArea(Circle c,P p1,P p2) {
    ld a1=PtoAngle(c,p1),a2=PtoAngle(c,p2);
    ld theta=fabsl(a1-a2);
    if (theta>PI) theta=2*PI-theta;
    return .5L*sq(c.r)*(theta-sinl(theta));
}
```

# 34 34-GEOM-Polygon

```
// POLYGON
using Poly=vector<P>;

// Signed area (positive if CCW)
ld signedArea(const Poly& v) {
    ld res=0;
```

```
    int n=v.size();
    for (int i=0;i<n;i++) res+=v[i].cross(v[(i+1)%n]);
    return res/2;
}

ld area(const Poly& v) { return fabsl(signedArea(v)); }
bool isCCW(const Poly& v) { return signedArea(v)>EPS; }

// Centroid
P centroid(const Poly& v) {
    P res(0,0);
    ld sa=0;
    int n=v.size();

    for (int i=0;i<n;i++) {
        ld cr=v[i].cross(v[(i+1)%n]);
        sa+=cr;
        res=res+(v[i]+v[(i+1)%n])*cr;
    }
    return res/(3*sa);
}

// Point in polygon: +1 inside, 0 on boundary, -1 outside (O
    (n))
int pip(P q,const Poly& v) {
    int n=v.size(),wind=0;

    for (int i=0;i<n;i++) {
        P a=v[i],b=v[(i+1)%n];
        if (onSeg(q,a,b)) return 0;

        int s=sign((b-a).cross(q-a));
        if (a.y<=q.y) {
            if (b.y>q.y&&s>0) ++wind;
        } else {
            if (b.y<=q.y&&s<0) --wind;
        }
    }

    return wind?1:-1;
}

// Convex hull (Andrew's monotone chain, O(n log n))
// Returns CCW hull (without collinear points on edges by
    default)
Poly convexHull(vector<P> pts,bool keepCollinear=false) {
    int n=pts.size();
    sort(pts.begin(),pts.end());
    pts.erase(unique(pts.begin(),pts.end()),pts.end());
    n=pts.size();

    if (n<2) return pts;

    Poly hull;

    auto bad=[&](P a,P b,P c) {
        ld cr=(b-a).cross(c-a);
        return keepCollinear?cr<-EPS:cr<=EPS;
    };

    // lower
    for (int i=0;i<n;i++) {
        while (hull.size()>=2&&bad(hull[hull.size()-2],hull.
            back(),pts[i]))
            hull.pop_back();
        hull.push_back(pts[i]);
    }

    // upper
    int lo=hull.size();
    for (int i=n-2;i>=0;i--) {
        while ((int)hull.size()>lo&&bad(hull[hull.size()-2],
            hull.back(),pts[i]))
            hull.pop_back();
        hull.push_back(pts[i]);
    }

    hull.pop_back();
    return hull;
}

// Is polygon convex? (assumes CCW, no repeated points)
bool isConvex(const Poly& v) {
    int n=v.size();
    for (int i=0;i<n;i++)
        if (sign((v[(i+1)%n]-v[i]).cross(v[(i+2)%n]-v[(i+1)%n
            ]))<0)
            return false;
    return true;
}

// Point in CONVEX polygon (O(log n), polygon must be CCW)
// returns +1 inside, 0 boundary, -1 outside
int pipConvex(P q,const Poly& v) {
    int n=v.size();

    if (n==1) return q==v[0]?0:-1;
    if (n==2) return onSeg(q,v[0],v[1])?0:-1;

    P o=v[0];

    if (sign((v[1]-o).cross(q-o))<0) return -1;
    if (sign((v[n-1]-o).cross(q-o))>0) return -1;

    int lo=1,hi=n-1;

    while (lo+1<hi) {
        int mid=(lo+hi)/2;
        if (sign((v[mid]-o).cross(q-o))>=0) lo=mid;
        else hi=mid;
    }

    ld cr=(v[lo+1]-v[lo]).cross(q-v[lo]);
    if (sign(cr)<0) return -1;

    if (sign(cr)==0||q==o||onSeg(q,o,v[1])||onSeg(q,o,v[n-1])
        )
        return 0;

    return 1;
}

// Polygon clipping by half-plane {left of directed line ab}
    (Sutherland-Hodgman)
Poly clipPolygon(Poly poly,P a,P b) {
    Poly res;
    int n=poly.size();

    for (int i=0;i<n;i++) {
        P c=poly[i],d=poly[(i+1)%n];
        bool ic=sign((b-a).cross(c-a))>=0;
        bool id=sign((b-a).cross(d-a))>=0;

        if (ic) res.push_back(c);

        if (ic!=id) {
            P inter;
            lineIntersect(Line(a,b),Line(c,d),inter);
            res.push_back(inter);
        }
    }

    return res;
}

// Polygoncircle intersection area
ld sectorArea2(P a,P b,ld r) {
    return .5L*r*r*atan2l(a.cross(b),a.dot(b));
}
```

```
ld polyCircleArea(const Poly& poly,Circle C) {
    int n=poly.size();
    ld res=0;

    for (int i=0;i<n;i++) {
        P a=poly[i]-C.c;
        P b=poly[(i+1)%n]-C.c;
        ld r=C.r;

        P p1,p2;
        int cnt=lineCircle(Line(a,b),Circle(P(),r),p1,p2);

        vector<P> pts={a};

        if (cnt==2) {
            auto onS=[&](P x) {
                return sign((b-a).dot(x-a))>=0&&sign((b-a).dot
                     (b-x))>=0;
            };

            if (onS(p1)) pts.push_back(p1);
            if (onS(p2)) pts.push_back(p2);

            if (pts.size()==3&&(pts[1]-a).norm()>(pts[2]-a).
                 norm())
                swap(pts[1],pts[2]);
        }

        pts.push_back(b);

        for (int j=0;j+1<(int)pts.size();j++) {
            P u=pts[j],v=pts[j+1],mid=(u+v)/2;

            if (mid.norm()<=r*r)
                res+=u.cross(v)/2;
            else
                res+=sectorArea2(u,v,r);
        }
    }

    return fabsl(res);
}

// Diameter of convex polygon (rotating calipers, O(n))
ld convexDiameter(const Poly& h) {
    int n=h.size();
    if (n==1) return 0;
    if (n==2) return dist(h[0],h[1]);

    ld res=0;

    for (int i=0,j=1;i<n;i++) {
        while ((h[(i+1)%n]-h[i]).cross(h[(j+1)%n]-h[j])>0)
            j=(j+1)%n;

        res=max(res,max(dist(h[i],h[j]),dist(h[(i+1)%n],h[j])
             ));
    }

    return res;
}

// Minkowski sum of two CCW convex polygons
Poly minkowski(Poly P1,Poly P2) {
    // rotate so first vertex is bottom-left
    auto reorder=[](Poly& v) {
        int idx=min_element(v.begin(),v.end())-v.begin();
        rotate(v.begin(),v.begin()+idx,v.end());
    };

    reorder(P1);
    reorder(P2);

    P1.push_back(P1[0]);
    P1.push_back(P1[1]);
    P2.push_back(P2[0]);
    P2.push_back(P2[1]);

    Poly res;
    int i=0,j=0,n=P1.size()-2,m=P2.size()-2;

    while (i<n||j<m) {
        res.push_back(P1[i]+P2[j]);

        P e1=P1[i+1]-P1[i],e2=P2[j+1]-P2[j];
        ld cr=e1.cross(e2);

        if (cr>EPS) i++;
        else if (cr<-EPS) j++;
        else { i++; j++; }
    }

    return res;
}

// Perimeter of polygon
ld perimeter(const Poly& v) {
    ld res=0;
    int n=v.size();

    for (int i=0;i<n;i++)
        res+=dist(v[i],v[(i+1)%n]);

    return res;
}
```

## 35 35-Treap

```
struct Node {
  int key, id, sz;
  unsigned pri;
  Node *l, *r;

  Node(int key, int id = 0)
      : key(key), id(id), sz(1), pri(rng()), l(0), r(0) {}

  static unsigned rng() {
    static mt19937 rng(chrono::steady_clock::now().
         time_since_epoch().count());
    return rng();
  }
};

int sz(Node *t) {
  return t ? t->sz : 0;
}

void pull(Node *t) {
  if(t)
    t->sz = 1 + sz(t->l) + sz(t->r);
}

bool leq(Node *t, int key, int id) {
  return t->key < key || (t->key == key && t->id < id);
}

void split(Node *t, int key, int id, Node *&a, Node *&b) {
  if(!t)
    return a = b = 0, void();

  if(leq(t, key, id)) {
    split(t->r, key, id, t->r, b);
    a = t;
  } else {
    split(t->l, key, id, a, t->l);
    b = t;
  }
```

```
  pull(t);
}

Node* merge(Node *a, Node *b) {
  if(!a || !b)
    return a ? a : b;

  if(a->pri > b->pri) {
    a->r = merge(a->r, b);
    pull(a);
    return a;
  }

  b->l = merge(a, b->l);
  pull(b);
  return b;
}

void insert(Node *&t, Node *x) {
  if(!t)
    return t = x, void();

  if(x->pri > t->pri) {
    split(t, x->key, x->id, x->l, x->r);
    t = x;
  } else if(leq(x, t->key, t->id)) {
    insert(t->l, x);
  } else {
    insert(t->r, x);
  }

  pull(t);
}

void erase(Node *&t, int key, int id) {
  if(t->key == key && t->id == id) {
    t = merge(t->l, t->r);
    return;
  }

  if(leq(t, key, id))
    erase(t->r, key, id);
  else
    erase(t->l, key, id);

  pull(t);
}
```